GPU Sharing Mechanisms for Multi-Tenant Inference Serving: An Evidence-Based Operating Systems Engineering Review

Muhammad Burhan Un Nadeem (SE-24071)
Muhammad Faaiz Alam (SE-24094)
Ali Raza (SE-24302)

Group A — Operating Systems Complex Engineering Problem
NED University of Engineering and Technology
17 October 2026

Abstract

Sharing an accelerator among inference tenants can improve useful capacity, yet contention changes latency and the meaning of isolation depends on the mechanism. We review temporal scheduling, NVIDIA Multi-Process Service (MPS), Multi-Instance GPU (MIG), and API-level remoting or virtualization with single-device inference evidence as the technical unit. A submitted protocol guides four publisher-platform searches and source-anchored screening. The current 43 accepted primary studies are a working set, not a completed final systematic selection: 45 extension reports await assessment, 96 earlier reports have unresolved retrieval, and four record links await adjudication. We distinguish same-experiment effects from cross-paper interpretation, and separate throughput, SLO goodput, latency percentiles, resource separation, security and fault containment. The evidence supports conditional choices: temporal control for strict priority when its scheduling constraints fit; tuned MPS for packing compatible inference; MIG for explicit partitioning where hardware, profiles and memory fit; and API control when interception and data movement fit the latency budget. Hybrids require local validation. These recommendations must be checked against the completed selection and missing configuration fields before submission.

Status and marking convention. Red text denotes an open decision, a provisional count, or an interpretation to revisit. Black study observations are bounded to the cited source and its extracted experiment; they do not make the review final. The companion `cep_report.txt` retains red passages as `[RED: ...]` markers. This manuscript is a substantive CEP draft for review, not a claim that the unresolved PRISMA flow is complete.

Contents

1 Introduction and engineering problem **3**

2 Review protocol and search execution **3**
2.1 Protocol, eligibility and review procedure 3
2.2 Databases, strings and counts 4
2.3 Selection status and PRISMA 2020 4

3 Evidence base, extraction and comparability **6**

4 Mechanism taxonomy and RQ1 **6**
4.1 Temporal admission, switching and priority 6
4.2 MPS and software spatial quotas 7
4.3 MIG and combinations 7
4.4 API remoting, virtualization and runtime control 7

5 Metrics, operating conditions and RQ2–RQ3 **8**

6 Comparative evidence matrix and RQ4 **8**
6.1 Trade-offs and conflicting evidence 9

7 Answers to RQ5 and engineering recommendation **10**

8 Research gaps and threats to validity **11**

9 Conclusion **11**

A Current primary-study register and quality scores **12**

B Source-anchored evidence and limitations **16**

C Completion and audit register **26**

1 Introduction and engineering problem

In a multi-tenant inference server, requests from distinct models or clients compete for execution slots, accelerator memory, caches and memory bandwidth. Exclusive GPU allocation simplifies attribution of latency to a tenant but can leave capacity idle when individual jobs are small or bursty. Concurrent execution can raise aggregate useful work while delaying a high-priority request, especially when the co-tenant's kernel or transfer pattern is unpredictable. Selecting a sharing mechanism is therefore an operating-systems decision about admission, scheduling, resource boundaries and observability, rather than a single throughput benchmark.

The assigned alternatives operate at different layers. Time-slicing switches or queues work over time; MPS enables multi-process execution and configurable active-thread shares; MIG presents hardware-defined GPU instances on supported NVIDIA devices; API remoting or virtualization routes calls, context state or data through an additional control path. Research systems frequently combine these with batching, model placement, kernel interception, memory management and feedback control. Such combinations cannot be evaluated as if one toggle alone produced the reported gain. The unit of our central comparison is concurrent inference on one physical GPU; multi-GPU outcomes are context unless a local sharing experiment can be identified.

We ask five analytical questions. **RQ1:** Which sharing/control mechanisms and boundaries have been evaluated for multi-tenant inference? **RQ2:** Which utilization, density, latency, fairness and isolation outcomes are actually measured, and which are comparable? **RQ3:** Under what workload, hardware and configuration conditions do results change? **RQ4:** What trade-offs and contradictory evidence emerge, and what experimental differences may explain them? **RQ5:** Which choice is defensible under stated requirements and evidence limits? The answers are synthesized in Sections 5–10, rather than determined by counting papers in each category.

2 Review protocol and search execution

2.1 Protocol, eligibility and review procedure

The submitted `SLR_protocol.tex/pdf` is the baseline. Its Group A execution record and dated amendments are in `groups/group-a/protocol.md` and `evidence/protocol_amendments.md`. The literature cutoff was extended to 11 October 2026. The study must be an English-language, peer-reviewed original full journal or conference report with relevant concurrent inference evidence and adequate technical detail; mixed inference/training work is used only where the inference result is separately identifiable. Training-only, cluster-only, editorial, survey and vendor-only material does not enter the primary set. A report-version link does not create an additional underlying study. Source retrieval failure remains an access outcome, not an eligibility exclusion.

One AI reviewer carried out the recorded screening and quality appraisal under the user's instruction. This is a limitation, not a claim of independent student consensus. Stable record, report and study IDs, reasons and extraction anchors are retained in the supporting workbook and CSV files. Quality uses ten criteria scored 0, 0.5 or 1, yielding a total out of 10; it assesses reporting and experimental adequacy, not proof of a mechanism's causal superiority. A human audit of screening, quality scoring and the remaining reports is still needed.

2.2 Databases, strings and counts

The NED E.A. Khan Library E-Resources listing motivated IEEE Xplore, Springer Nature Link, ACM Digital Library and ScienceDirect. The exact platform-specific Boolean strings, filters, interface URLs, dates and export filenames are preserved in `search_runs.csv` and `search_runs_received.csv`; the executable strings are too long to replace with a falsely standardized shorthand here. The common concept combines GPU with inference/model serving terms and the relevant temporal, MPS, MIG or virtualization family. IEEE used four family searches; Springer and ACM used broader master searches; ScienceDirect used four family searches after the master exceeded the interface connector limit. The ScienceDirect split also dropped DNN/LLM synonyms from the common block, a documented recall limitation. Supplementary discovery is identified separately and Google Scholar is not treated as a formal database.

The 11 October IEEE/Springer capture has 689 formal occurrences (102 IEEE, 587 Springer). The supplied ACM/ScienceDirect exports have 3,727 parsed occurrences (1,829 ACM, 1,898 ScienceDirect), for 4,416 parsed formal occurrences across the four sources. ACM displayed 1,600 or 1,603 hits in different saved observations while exports contain 1,829 rows; ScienceDirect MPS displayed 517 in an earlier note and 563 upon later reopening. These differences are preserved rather than silently normalized. A displayed hit count, parsed export row and unique report have different denominators. No publisher entitlement or complete coverage is inferred from an E-Resources listing.

2.3 Selection status and PRISMA 2020

Figure 1 is an interim stage map, with arithmetic cross-checked against `interim_four_source_prisma.json`. It is deliberately labelled as provisional: four additional links, row-level Step 3–5 decisions, report versions and retrieval statuses still need canonical reconciliation. The 11 October 21-study flow is a separate historical snapshot, and batch additions must not be pasted into its boxes. The 43-study current set combines 21 historical accepted studies with 10, 3 and 9 accepted in three assessed batches. Two earlier eligible reports remain pending formal integration. The queue contains 45 unassessed reports, and 96 older reports remain unresolved for retrieval. The final PRISMA 2020 diagram and final unique-study count cannot yet be certified.

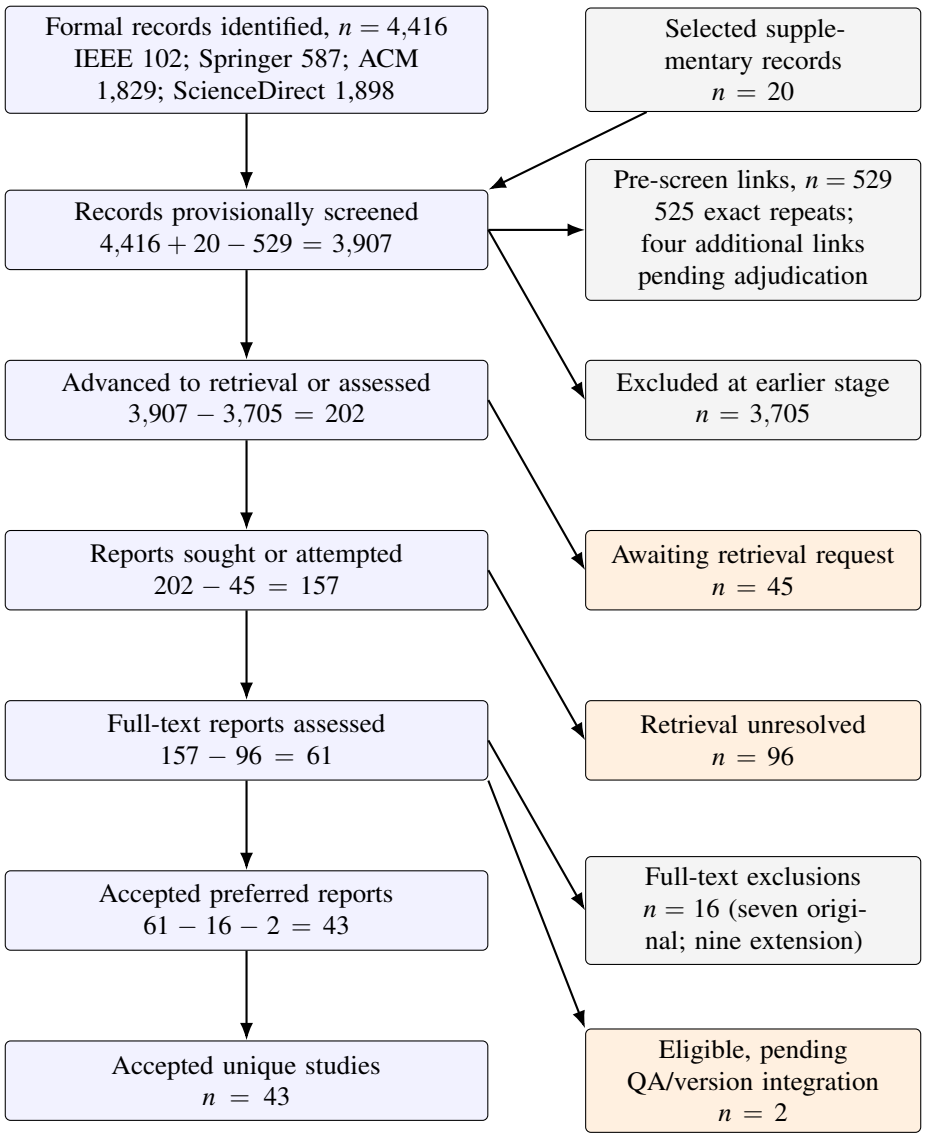


Figure 1: Interim PRISMA 2020 stage map for the four captured database sources and selected supplementary records, 17 October 2026. The 529 pre-screen links include four Step 5 links awaiting canonical record/report adjudication; the 3,907 screening count is provisional. The 45 unsought reports and 96 unresolved retrieval cases are open, and two eligible reports await formal QA and version integration. Counts represent parsed occurrences, not verified search coverage; publisher hit/export discrepancies remain open. Source arithmetic and limitations: `interim_four_source_prisma.json`.

The identities are 4,416+20-525-4=3,907; 3,705+202=3,907; 45+157=202; 96+61=157; and 16+2+43=61. These equations demonstrate arithmetic consistency of the interim map, not that each link, screening event or source coverage has been fully audited. The historical 21-study figure remains a separate checkpoint in the query report; this interim map must not be submitted as a final PRISMA flow.

3 Evidence base, extraction and comparability

The working set contains 43 distinct accepted study IDs, each with a preferred report, a quality row and a bounded extraction row. The bibliography preserves those report keys. The complete study register in Appendix A gives the title, year, mechanism and quality score; Appendix B maps a concise extracted observation and limitation to the source location. The workbook holds fuller version and screening records. Peer-reviewed status of a venue does not validate all figures or indicate that our team reran its experiments. Some source copies are Markdown conversions with interleaved columns or inaccessible plot pixels; untranscribed values are never interpolated.

We distinguish *request throughput* (all processed requests per unit time) from *SLO goodput* (requests meeting a specified deadline), GPU busy time, SM occupancy, instance count, energy and a paper's custom utilization/efficiency ratio. A lower instance count in a fleet experiment is not a directly measured local GPU occupancy gain. Mean latency, p90/p95/p98/p99, time to first token (TTFT), time between tokens (TBT), job completion time (JCT), and violation rates have different meanings. A paper's p99 under one arrival process cannot be numerically pooled with another paper's mean under a different model, GPU or baseline. InferFair's unfairness ratio is also distinct from security isolation. The evidence matrix therefore uses within-study comparisons and explicitly marks indirect cross-study recommendations.

For an interpretable comparison one must know the GPU SKU and memory, driver/runtime, MIG profile or MPS active-thread quota, model and input/output sizes, batching, tenant mix, arrival trace, SLO definition, comparator resource budget, warm-up and measurement windows, and uncertainty estimate. Some of these parameters remain NOT EXTRACTED in individual records, so a formal effect-size meta-analysis is not justified. Direct security and injected-fault tests are scarce; interference reduction, resource separation, adversarial confidentiality and fault containment are separate properties.

4 Mechanism taxonomy and RQ1

4.1 Temporal admission, switching and priority

Clockwork schedules predictable GPU work with SLO-aware goodput as a central outcome [1]. PipeSwitch explores pipelined context switches [2]. REEF, Sirius, Orion, XSched, Paella, SPLIT, Miriam, SwitchFlow and ElasticRoom use different combinations of preemption, priority, kernel or subgraph control, compilers and feedback [3, 4, 5, 6, 7, 8, 9, 10, 11]. These are not interchangeable implementations of a generic time slice: a scheduler that can intercept launches, preempt selected kernels and control memory handover has a different cost and failure mode from exclusive model execution. SwitchFlow's concurrent inference evidence is in its inference-specific experiment; its larger mixed inference/training p95 example is a separate stratum [10].

Temporal control is attractive when a small high-priority request should interrupt or avoid best-effort interference. It still needs admitted kernels to finish or be preempted in time, enough memory for models and a credible worst-case workload profile. More aggressive interruption can lower throughput of the best-effort tenant. A priority guarantee is a performance property under evaluated arrivals, not an adversarial security boundary. The strength of this inference is direct for the cited within-study experiments and indirect when generalized to a new GPU or model.

4.2 MPS and software spatial quotas

MPS enables simultaneous work and configurable active-thread limits; research systems use it to provision GPUlets, adapt quotas, right-size kernels or shape interference [12, 13, 14, 15, 16, 17, 18]. Bless and DeInfer add scheduling or prediction, so their gains cannot be assigned to MPS alone [19, 18]. A quota may limit some compute contention while cache, memory traffic and process behavior remain shared. VastPipe's A100 pair and separate AMD CU-mask experiments must not be pooled; the latter is not NVIDIA MPS [14]. DeInfer directly observes remaining latency interference under MPS spatial allocation [18].

Provisioning can improve density when models are small enough and arrivals permit overlap, but allocation needs tuning against model shape, batch and peak load. GSLICE's shadow-instance load time is a transition cost; its GUE is not a generic GPU-busy percentage [13]. iGniter's initial p99 violation before shadow-process recovery qualifies its later operating point [15]. MPS supports an engineering control for capacity but the extracted experiments do not establish it as a memory-security or fault-isolation guarantee.

4.3 MIG and combinations

MIG allocates hardware-defined GPU instances on supported devices. It can give explicit resource partitions, but a slice is finite: model weights, batch memory and number of concurrently provisioned instances must fit. GNN-Triton reports thrashing beyond the local slice's model capacity [20]. Dyna-P shows that more smaller partitions can increase concurrent jobs while increasing mean JCT [21]; that result does not measure online p99. ParvaGPU, MIGER, MMK, BOER, FluidFaaS and Protean add MPS, grouping, placement, migration or serverless orchestration [22, 23, 24, 25, 26, 27]. Their whole-system figures include scheduling decisions in addition to local partition behavior. MIGServe's four-GPU sustainable rate at 90% SLO attainment is not a one-GPU p99 measurement [28].

Hybrid packing is conditional. BOER identifies a local case where a ParvaGPU-style hybrid underperforms MIG-only serving for larger models, while helping a light MobileNet case [25]. EnerTune's energy-aware placement shows that the most utilized pairing need not minimize energy, and even its favorable local energy pairing changes latency [29]. MIG partitioning does not automatically establish that every security or fault-containment property was tested in these papers. Exact supported profiles, reconfiguration disruption and threat model need to be checked against the target hardware before a final deployment recommendation.

4.4 API remoting, virtualization and runtime control

An API layer can intercept or route calls, isolate runtime objects, schedule kernels or move contexts/data across boundaries. Torpor gives a directly measured warning: a BERT example

reports native 42 ms, remote 43 ms, PCIe swap 144 ms and NVLink swap 45 ms in its Table 4 [30]. These are the paper's setting, not universal interface overheads. StreamBox, KRYPTON, Tally, Orion, XSched, ArkGPU and SGDRC differ substantially in intercepted APIs, process assumptions, memory behavior and scheduling policy [31, 32, 33, 5, 6, 34, 35]. An API wrapper changes the trusted implementation surface and can impose transfer, copying, synchronization or compatibility costs. KRYPTON contains a specific fault-propagation test; one experiment cannot be generalized to all wrappers [32]. StreamBox's sandbox separation likewise should not be promoted to proof of adversarial security [31].

5 Metrics, operating conditions and RQ2–RQ3

Several studies report measured improvements that are deliberately non-comparable across strata. REEF's GV100 PyTorch DISB-C experiment compares normalized p99 and separately normalized throughput against Orion; a TensorRT stratum gives a throughput counterexample [3]. InferFair's local Table 3 reports a goodput and slowdown-fairness result under a particular load ratio, with smaller differences at other ratios [36]. AdMix's local TP1 and TP2 experiments use distinct LLM models and device counts; their best-effort throughput ratios are not pooled [17]. The SoCC interference study measures P90 TBT under synthetic co-tenants and finds that separated SM sets still share cache and memory-bandwidth pressure [37]. Hardware partitions and request-level isolation therefore require separate measurements.

Edge GPU studies add useful control ideas but limited direct transfer to data-center MIG/MPS. Miriam coordinates elastic kernels on edge GPUs lacking the server mechanisms; its paper's upper-bound throughput and critical-task overhead are paired outcomes under its evaluated mix [9]. SPLIT evaluates model-block scheduling and violations on one edge GPU [8]. Edge-sky couples GPU, CPU and Edge TPU placement, so its combined throughput is not a pure GPU-sharing effect [38]. WattWiser uses four AMD MI50 devices for placement and power; local shared pairs inform latency while server power is a wider result [39]. ITIF's shared transformer backbone changes computation itself, so its gain is not a scheduler-only comparison [40].

Arrival shape and model composition influence the winner. FaST-GShare's local eight-pod RNNT setting differs from a single racing pod; the model, share and tail threshold define its result [16]. Deft shows that synchronization frequency selected for one batch or image size can inflate another job's completion time after a workload change [41]. EnerTune's power/frequency and batching settings change the resource objective [29]. FluidFaaS and ESG report workflow or system SLO effects that combine local GPU sharing with pipeline design and placement [26, 42]. These reports remain relevant to mechanism deployment, but their totals do not identify the isolated gain from a partition or quota.

6 Comparative evidence matrix and RQ4

Table 1: Conditional comparison; each cell describes a supported design consideration, not a universal ranking.

Mechanism	Useful condition / evidence	Latency and density trade-off	Boundary and important limitation
Temporal control	Prioritized, bursty requests when scheduler can control or preempt work [1, 3, 6, 8].	Priority p99 or SLO may improve while best-effort throughput falls [3, 33]; memory handover and transition cost remain.	Software scheduling controls execution order; it does not by itself prove memory confidentiality or fault containment.
MPS/quotas	Compatible small models, concurrent execution and measured quota tuning [12, 13, 14, 18].	Density can improve; oversubscription, load transients and shared bandwidth may harm tails [15, 37].	Active-thread shares do not isolate all resources; security and fault claims require direct tests.
MIG/partitions	Supported device, well-sized models and explicit instance resource budgets [20, 21].	Profile fragmentation, memory fit and reconfiguration can reduce usable density or increase JCT.	More explicit hardware resource boundary; a cited performance experiment is not an adversarial security evaluation.
MIG+MPS hybrids	Model grouping and profile/quota search with monitoring [22, 23, 25].	Light models may benefit; larger models may lose to MIG-only [25]; optimizer cost matters.	Multiple controls and service-level placement confound attribution; evaluate each local stratum.
API control/remoting	Interception needed for fine-grained scheduling or process multiplexing [30, 32, 5].	Transfer fabric and context path can dominate; PCIe versus NVLink differs within Torpor [30].	Compatibility, shared process assumptions and trusted wrapper matter; only targeted faults were evaluated.

6.1 Trade-offs and conflicting evidence

Priority latency versus background throughput is an observed conflict rather than a rhetorical possibility. Tally's 10% idle-time stratum reports normalized background throughput 1.19 versus 1.65 for TGS, even though its policy targets protected inference latency [33]. REEF's PyTorch priority result and its separate TensorRT throughput comparison show that the conclusion changes with runtime and denominator [3]. We infer that a latency-first controller may knowingly leave capacity or best-effort progress on the table; the size of that sacrifice must be measured at

the target arrival rates. This does not establish Tally or REEF as globally better or worse.

Packing versus interference is another genuine conflict. VastPipe reports local pair gains over default MPS in its A100 configuration [14]; DeInfer observes that MPS limits still permit shared-resource latency interference [18]. The SoCC Green Context experiments separate SM sets but not L2 and off-chip bandwidth, with substantial P90 TBT sensitivity under a synthetic copy or sleep co-tenant [37]. Their models, GPU generations, metrics and intervention layers differ. We infer that quota-only or SM-only allocation should be accompanied by per-tenant tail measurements and a memory-traffic stress case, rather than that the papers disagree on one controlled experiment.

Partition density versus model fit is visible in GNN-Triton's local thrashing threshold and Dyna-P's concurrency/JCT tension [20, 21]. BOER's larger-model counterexample further prevents "MIG+MPS always beats MIG" [25]. MIGServe and FluidFaaS evaluate broader server systems, so their rates and SLO hit percentages cannot refute a one-GPU memory-capacity result [28, 26]. The amount of device memory, model weights, batch, chosen instance layout and migration policy are plausible effect modifiers, not measured universal coefficients.

Resource utilization versus energy is a separate objective. EnerTune's local SDXL/MobileNetV3 pairing consumes less energy than no sharing while increasing latency in that example [29]. WattWiser's aggregate power outcome concerns four AMD GPUs and placement [39]. Neither demonstrates a universal energy benefit from every MIG profile or from simply increasing GPU busy time. The objective function should include electricity, SLO penalties and the cost of extra devices when those matter.

7 Answers to RQ5 and engineering recommendation

For a strict per-tenant latency SLO and a high-priority versus best-effort mix, begin with exclusive admission or a temporal controller whose measured preemption, handover and model-fit behavior satisfy the target percentile. REEF, Clockwork, Sirius, XSched and related experiments support the possibility of protecting priority work in their own settings [3, 1, 4, 6]. Reserve capacity or accept lower background throughput when necessary; Tally supplies a direct caution [33]. This is an inference from heterogeneous studies, not a certified SLO for a new GPU. A final target workload trace, percentile and allowed throughput loss are needed to make the deployment choice concrete.

For density with compatible small models and flexible SLOs, trial MPS with explicit per-model caps and interference monitoring, using solo and concurrent baselines at realistic burst rates [12, 13, 14, 18]. Report achieved goodput and tail latency alongside density. If memory footprint or cache/bandwidth contention dominates, a cap alone is insufficient. If supported hardware and model fit permit it, trial MIG partitions for more explicit allocation; test several profiles and the cost of reconfiguration, then compare MIG-only and MIG+MPS on the same service mix [20, 21, 25]. BOER's adverse hybrid case is a reason to retain the MIG-only baseline.

Choose API remoting or virtualization when it supplies required fine-grained control or process integration and measured call, transfer and context overhead fits the SLO. Benchmark the actual PCIe/NVLink path, tensor sizes and client/runtime compatibility. Torpor illustrates why fabric selection changes latency [30]. A security or fault-containment requirement additionally calls for a specified threat model and direct testing; performance-interference control cannot substitute

for that evidence. No cross-mechanism adversarial security ranking is justified by the current extraction.

Use an explicit decision record: target GPU and supported features; peak and burst arrival trace; model and batch memory; p95/p99 or TTFT/TBT SLO; minimum goodput and tenant count; allowed reconfiguration interruption; energy or hardware-cost objective; security/fault threat model; then a same-device A/B comparison with solo and shared baselines. A mechanism is acceptable only if it meets the hard constraint in that measured setting; choose among acceptable candidates using the stated secondary objective. The comparison must disclose whether a whole-server controller, compiler, batching policy or workflow scheduler also changed. This procedure is our engineering interpretation of the evidence, not a benchmark outcome reported by one paper.

8 Research gaps and threats to validity

The working extraction shows incomplete apples-to-apples comparisons of time slicing, MPS, MIG and remoting on one GPU under the same models, arrivals, SLOs, resource budget and driver/runtime. Tail and fairness metrics are inconsistent. Several systems combine local sharing with batching or global placement, and some energy totals are whole-server rather than per-partition. Direct fault-injection and adversarial isolation studies are sparse relative to latency experiments. Recent LLM decoding introduces TTFT/TBT and memory-bandwidth behavior that does not map neatly to older image-classifier throughput. These gaps follow from the recorded comparison fields and negative cases; the final gap inventory is subject to the remaining full texts and parameter extraction.

Review-level threats include incomplete formal export coverage, the ACM displayed/export mismatch, ScienceDirect filter ambiguity, an amended cutoff, a single AI reviewer, rule-assisted title exclusions, 45 unassessed extension reports, 96 unresolved retrieval cases and two pending eligible reports. The interim four-source counts balance algebraically but not yet at every canonical report/version row. Published studies favor systems with favorable evaluations; negative findings and inaccessible work may be underrepresented. Some Markdown conversions lose figure pixels or interleave columns; we retain labelled prose and tables with locations, do not interpolate plots, and label untranscribed parameters. Quality totals assess reporting, and a threshold alone cannot resolve indirectness or missing experiments. A final sensitivity analysis after screening and human review is required before calling the synthesis complete.

9 Conclusion

Current primary evidence supports conditional engineering selection rather than a universal winner. Temporal admission and preemption can protect prioritized inference with throughput or integration costs. Tuned MPS can pack compatible models but retains shared-resource interference. MIG exposes defined partitions with profile, memory and reconfiguration costs; hybrid gains depend on model size and placement. API control can add fine-grained scheduling while transfer and compatibility impose workload-specific overhead. Performance protection, memory separation, adversarial security and fault containment must be evaluated independently. The 43-study working set and interim PRISMA map must be reconciled with all pending retrieval, screening and version decisions before this becomes the final submitted SLR.

A Current primary-study register and quality scores

This appendix is generated from `current_extraction.csv` and `current_quality.csv`; each ID denotes one preferred report/study in the working set. Scores are out of 10, and the source/venue and review caveat are retained in the evidence workbook. A score does not establish a measured security property.

Table 2: The 43-study current accepted set. Mechanism phrases are condensed from the extraction register.

ID	Preferred study	Mechanism	QA
S001	Real-time, Work-conserving GPU Scheduling for Concurrent DNN Inference (2025) [3]	Reset-based temporal preemption + dynamic kernel padding; NVIDIA binary instrumentation	8.5
S002	Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing (2022) [12]	MPS + temporal	8.5
S003	PipeSwitch: Fast Pipelined Context Switching for Deep Learning Applications (2020) [2]	Temporal context switching	9.0
S004	Serving DNNs like Clockwork: Performance Predictability from the Bottom Up (2020) [1]	Temporal exclusive scheduling	8.5
S005	Torpor: GPU-Enabled Serverless Computing for Low-Latency, Resource-Efficient Inference (2025) [30]	API remoting + temporal swapping	8.5
S006	StreamBox: A Lightweight GPU SandBox for Serverless Inference Workflow (2024) [31]	API/runtime sandbox virtualization	8.0
S007	KRYPTON: Efficient Performance-Aware GPU Sharing with Compatibility and Isolation through Kernel Space Interception (2025) [32]	Kernel API virtualization + hybrid	9.0
S008	SIRIUS: Colocating ML Inference and Training with Fast GPU Memory Handover (2025) [4]	Temporal + MPS memory handover	8.5
S009	Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads (2025) [33]	CUDA API interception + priority	9.0
S010	ParvaGPU: Efficient Spatial GPU Sharing for Large-Scale DNN Inference in Cloud Environments (2024) [22]	MIG + MPS	8.0
S011	Orion: Interference-aware Fine-grained GPU Sharing for ML Applications (2024) [5]	CUDA API interception + temporal	9.0
S012	GSLICE: Controlled Spatial Sharing of GPUs for a Scalable Inference Platform (2020) [13]	MPS adaptive provisioning	9.0

ID	Preferred study	Mechanism	QA
S013	KRISP: Enabling Kernel-wise RIght-sizing for Spatial Partitioned GPU Inference Servers (2023) [43]	Spatial partition/API control (AMD)	8.0
S014	XSched: Preemptive Scheduling for Diverse XPUs (2025) [6]	Temporal API interposition	9.0
S015	Paella: Low-latency Model Serving with Software-defined GPU Scheduling (2023) [7]	Temporal software scheduling + MPS comparison	8.5
S016	Optimizing High-Throughput Inference on Graph Neural Networks at Shared Computing Facilities with the NVIDIA Triton Inference Server (2024) [20]	MIG + concurrent model serving	8.5
S017	VastPipe: A High-Throughput Inference System via Adaptive Space-Division Multiplexing for Diverse Accelerators (2025) [14]	MPS adaptive quotas; separate AMD CU-mask stratum	6.5
S018	ArkGPU: enabling applications' high-goodput co-location execution on multitasking GPUs (2023) [34]	CUDA API interception + NVML monitoring and admission throttling	7.0
S019	Dyna-P: placement-aware dynamic partitioning for lightweight applications with modern GPUs (2025) [21]	MIG dynamic placement/reconfiguration with reconfiguration-last policy	7.0
S020	iGniter: Interference-Aware GPU Resource Provisioning for Predictable DNN Inference in the Cloud (2023) [15]	MPS active-thread quotas + interference-aware batch/provisioning and shadow process	8.5
S021	InferFair: Towards QoS-aware scheduling for performance isolation guarantee in heterogeneous model serving systems (2024) [36]	Temporal SLO-aware scheduling + feedback goodput balancing; MPS comparator	7.5
S022	MIGER: Integrating Multi-Instance GPU and Multi-Process Service for Deep Learning Clusters (2024) [23]	MIG partitions with per-partition MPS online/offline co-location, local partition planner and fragmentation control	8.0

ID	Preferred study	Mechanism	QA
S023	Towards Optimal Preemptive GPU Time-Sharing for Edge Model Serving (2024) [41]	Deft: adaptive CUDA synchronization frequency controlling kernel queues and priority preemption; API interposition prototype	7.0
S024	FaST-GShare: Enabling Efficient Spatio-Temporal GPU Sharing in Serverless Computing for Deep Learning Inference (2023) [16]	FaST-GShare: MPS spatial shares plus CUDA interception/token-based temporal quotas; CUDA IPC model sharing	7.5
S025	SPLIT: QoS-Aware DNN Inference on Shared GPU via Evenly-Sized Model Splitting (2023) [8]	SPLIT: genetic offline model-block splitting with token-based greedy temporal preemption	8.5
S026	MIGServe: Layout-Aware Multi-Instance GPU Management for Efficient LLM Serving (2026) [28]	MIGServe: buddy-aware MIG placement, proactive pair migration and temperature-guided eviction	8.0
S027	MMK: A Hybrid Scheduling Framework for Fine-Grained GPU Sharing for Deep Learning Applications (2026) [24]	MMK: MIG, elastic MPS oversubscription and CUDA-intercepted kernel-priority scheduling	8.0
S028	BOER: Enhancing Resource Utilization for Deep Learning Inference with Hybrid Spatial GPU Sharing (2025) [25]	BOER: MIG model grouping with Bayesian-optimized MPS configurations inside partitions	8.5
S029	Adaptive GPU Sharing for Real-time LLM Serving with Best-effort Workloads (2026) [17]	AdMix: adaptive MPS active-thread caps, lightweight LLM estimator and best-effort pause/full-access controls	8.5

ID	Preferred study	Mechanism	QA
S030	SGDRC: Software-Defined Dynamic Resource Control for Concurrent DNN Inference on NVIDIA GPUs (2025) [35]	SGDRC: CUDA/TVM runtime with software VRAM-channel coloring, bimodal tensors and tidal SM masks	8.5
S031	Beyond Utilization: Energy-Conscious GPU Sharing for Inference Serving (2026) [29]	EnerTune: MIG allocation with joint frequency/batch selection, colocation-power model and runtime tuner; separate MPS portabi	9.0
S032	FluidFaaS: A Dynamic Pipelined Solution for Serverless Computing with Strong Isolation-based GPU Sharing (2025) [26]	MIG partitioned serverless functions with pipelined instance construction, eviction and dynamic resource assignment	8.0
S033	Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing (2025) [19]	Bless: adaptive kernel-squad scheduling, MPS contexts with SM affinity and temporal/spatial quota adjustments	8.0
S034	Understanding GPU Resource Interference One Level Deeper (2025) [37]	CUDA streams and CUDA Green Context SM partition for controlled co-location; interference characterization rather than a depl	8.0
S035	Reaching for the Sky: Maximizing Deep Learning Inference Throughput on Edge Devices with AI Multi-Tenancy (2023) [38]	Concurrent model execution and dynamic placement across edge device GPU/CPU and Edge TPU	7.0

ID	Preferred study	Mechanism	QA
S036	Miriam: Exploiting Elastic Kernels for Real-time Multi-DNN Inference on Edge GPU (2023) [9]	Elastic kernels and contention-aware coordination for mixed-criticality inference	8.0
S037	ElasticRoom: Multi-Tenant DNN Inference Engine via Co-design with Resource-constrained Compilation and Strong Priority Scheduling (2024) [11]	Resource-constrained compilation and priority scheduling with concurrent GPU inference	8.0
S038	SwitchFlow: Preemptive Multitasking for Deep Learning (2021) [10]	Preemptive subgraph scheduling and interleaved model executors	9.5
S039	ITIF: Integrated Transformers Inference Framework for Multiple Tenants on GPU (2023) [40]	Shared transformer backbone with operator-level multi-tenant execution	8.0
S040	Towards SLO-Compliant and Cost-Effective Serverless Computing on Emerging GPU Architectures (2024) [27]	Protean orchestrates MIG partitioning and MPS sharing for GPU serverless inference	8.5
S041	WattWiser: Power & Resource-Efficient Scheduling for Multi-Model Multi-GPU Inference Servers (2023) [39]	Sharing-aware placement and request routing to co-located models on physical GPUs	9.5
S042	DeInfer: A GPU resource allocation algorithm with spatial sharing for near-deterministic inferring tasks (2024) [18]	Interference-aware MPS spatial allocation with latency prediction	9.0
S043	ESG: Pipeline-Conscious Efficient Scheduling of DNN Workflows on Serverless Platforms with Shareable GPUs (2024) [42]	Pipeline-aware placement combining GPU sharing, batch size and SLO distribution	8.0

B Source-anchored evidence and limitations

The following condensed cards give one bounded finding and its corresponding caveat for every current study, with source location. Full extraction fields and source types are in the CSV/workbook; no new unverified measurement is added by this rendering.

Table 3: Bounded observation, qualification and source anchor for each accepted study.

ID	Extracted observation	Limitation and source location
S001 [3]	GV100 PyTorch DISB-C: Orion p99 normalized latency1.17x vs Reef below1.05x; normalized total throughput0.97x vs1.14x(Fig29). TensorRT comparison trades throughput: mean1.21x Triton vs1.11x Reef(Fig28). All ratios retain model-solo normalization; do not mix engines/AMD strata.	Explicit OSDI2022 extension; select journal as preferred report and retain GA-S001 once. Requires deterministic/stateless kernels; closed-source comparison omits padding. Memory assumed sufficient; bandwidth contention and input-shape profiling limits. Printed publication date differs from some issue-year indexing; recorded2025 follows supplied copy and is in cutoff either way. *Anchor:* Article pp1:23-1:24,1:31-1:34; sections6.2,6.6,7; Figs28-29; Markdown lines1111-1180,1600-1820
S002 [12]	Local gpulets support interference-aware provisioning; cluster throughput is a different outcome.	Server-level gains cannot be treated as one-GPU utilization. *Anchor:* PDF p10 Table2/3; section5; local profiling section3
S003 [2]	Fig7: inference preemption latency reduced by up to 38.2ms.	Average switch/inference latency is not p99; multi-VM throughput separate. *Anchor:* PDF pp11-13; sections6-6.2; Fig7
S004 [1]	Fig5 measures goodput satisfying SLO, rather than raw request throughput.	Cross-hardware INFaaS comparison is indirect; controller/worker distinction. *Anchor:* PDF p10; section6.1; Fig5
S005 [30]	BERT Table4: native42ms, remote43ms, PCIe swap144ms, NVLink swap45ms.	p98 tail; busy time is not SM occupancy; node scaling separate. *Anchor:* PDF pp10-11; Table4; sections7.2-7.3
S006 [31]	Table2 evaluates runtime sandbox workflows on shared GPUs.	Sandbox memory separation is not proof of adversarial security. *Anchor:* PDF p10 onward; section9; Table2
S007 [32]	Inference-specific sharing comparisons; section2.2 tests fault propagation.	Baseline adaptations and workload mix constrain attribution. *Anchor:* PDF p10 Tables1-2; section5; section2.2 Fig3

ID	Extracted observation	Limitation and source location
S008 [4]	Inference p99/SLO outcomes accompany training progress; separate TTFT/TBT LLM case.	TaskSwitch is a reimplementation, not vanilla PipeSwitch. *Anchor:* Section6.1 Fig10; section6.6
S009 [33]	At 10% idle time: normalized throughput Tally1.19 vs TGS1.65.	Latency protection sacrifices some BE throughput; author manuscript. *Anchor:* PDF pp9-11; sections5-5.3; Fig6
S010 [22]	Fig7 separates internal/external fragmentation; Fig5 counts GPUs.	Author manuscript; unusual runtime version; GPU count is not occupancy. *Anchor:* PDF p8 onward; sectionIV; TableIV; Figs5/7
S011 [5]	Interference-aware scheduling compares inference latency and BE throughput.	Prototype clients share a process; REEF-N differs from AMD REEF. *Anchor:* PDF pp8-9; sections5.3-6.1; Tables1/3
S012 [13]	GUE measures efficiency; shadow instance loading takes about12s in Fig15.	GUE is not GPU busy percentage; median latency not p99; academic mirror. *Anchor:* PDF pp8-12; section4; Table3; Fig15
S013 [43]	Kernel-wise right-sizing evaluates throughput and p95 inference latency.	Feature emulated; overhead subtracted; baseline is not NVIDIA MPS. *Anchor:* PDF pp9-10; sectionsV-VI; TableIII
S014 [6]	Fig15: relative p99 1.53/1.51/1.07 respectively versus solo.	Data assumed to fit memory; CUDA graphs and undocumented driver limits. *Anchor:* PDF pp14-15; section8 Case3 Fig15; section9
S015 [7]	p99 latency and throughput under bursty model mixes; Fig12 includes MPS.	Custom Triton TVM wrapper; MPS omitted from Fig11; dynamic batching unsupported. *Anchor:* PDF pp9-12; section7 Table3/Figs11-12; section8
S016 [20]	Fig10: thrashing after two models/20GB or five/40GB.	Local slice evidence only; multi-server scaling separate; isolation properties described, not attack-tested. *Anchor:* HTML Multi-model Scaling/Fig10; Triton implementation; limitations

ID	Extracted observation	Limitation and source location
S017 [14]	A100 VGG16+GoogLeNet: throughput 2.7x no MPS and 1.5x default MPS; not a universal average or p99 result. NLP/CV mean 2.43x no MPS, 1.45x default MPS. MI100 mean 1.89x no mask, 1.24x default mask.	Include only local GPU inference evidence, not the main 1000-NPU cluster results. Offline Q-table assumes relatively stable request mix; GPU-specific quotas, repetitions and variance unresolved. Chart bars are not preserved in Markdown. Preliminary ICCD2021 report is linked as an unassessed version lead. *Anchor:* Published pp457-459; sections6.7-6.9; Figs14-16; uploaded Markdown lines1150-1290
S018 [34]	Section5.2 reports all30 tested pairs satisfy the 2x-solo p99 QoS target with ArkGPU. Low-contention pairs can lose about5% QPS from throttling. Four-GPU app-mix goodput gains94.98% vs k8s-native and38.65% vs MPS are separate system-level results.	Mixed inference/training qualifies via explicit inference p99/QPS and workload transfer rationale; it does not establish inference-only performance. Polling/API sleeps can add delay; BE checkpoint/restart affects costs. Huge goodput ratios can have near-zero baseline yield. Repetitions/variance and chart coordinates unresolved. *Anchor:* Published pp313-319; sections4.3,5.1-5.4; Table3; Figs14-21; Markdown lines920-1360
S019 [21]	Single-A30 batch experiments: smaller partitions increase concurrency but can increase mean JCT; not online p99 evidence. Fig8 broader job set reports39.03% lower makespan vs StaCon,12.14% vs MISO and14.7% higher normalized STP vs StaCon. Prediction-error sensitivity:15% perturbation adds8.09% makespan.	Retain one-GPU inference strata separately from two-GPU aggregate and cluster simulation. JCT/makespan/STP cannot replace request-tail latency or goodput. Reconfiguration can stop/restart active jobs; neither architectural MIG claims nor OOM avoidance prove hostile isolation. Model labels/parameter counts and corrupted overhead fraction require caution. *Anchor:* PDF pp11-16; sections4.1-4.3.2; Table2; Figs8-13; Markdown lines700-865

ID	Extracted observation	Limitation and source location
S020 [15]	Local four-model latency prediction error1.53%-5.02%(Fig13). Fig14 deployment: iGniter6 GPUs/0 SLO violations vs gpu-lets+8/3,FFD+5/10,GSLICE+6/3;25% lower hourly instance cost vs gpu-lets+, not25% higher local utilization. Fig17 shows an initial p99 violation before shadow-process recovery.	Constant arrivals and four vision models limit bursty/LLM transfer. Negligible PCIe contention assumption can underpredict AlexNet; baselines modified. Steady-state/post-correction SLO success is not an unconditional guarantee; shadow standby incurs capacity/memory costs. Chart bars/latency traces unavailable in Markdown. *Anchor:* Published pp820-824; sections4.2,5.1-5.4; Figs13-18; Markdown lines938-1245
S021 [36]	Local Table3 load3:1:2: unfairness1.05 vs Clockwork1.50; SLO goodput524.29 vs457.24req/s. Other load ratios give smaller differences. Local unfairness is max/min model slowdown(Eq2), not p99 or security. Four-GPU MAF outcome kept separate.	Two-column Markdown interleaves algorithms/prose; use labelled Table3 rows and sections, not scrambled formulas. Repetitions/variance and software versions unresolved. MAF trace has minute-level invocation counts; cross-study batching and isolation definitions differ. *Anchor:* Published pp17-18; sections5.1-5.4; Table3; Eq2; Markdown lines718-899
S022 [23]	Section 3.1/Fig. 2 directly compares local MPS 57/29/14 percent and MIG 4g/2g/1g allocations. Section 5.2/Fig. 8 reports testbed mean JCT 46.6% below MuxFlow and 36% below MISO; these are job/cluster outcomes, not inference p99 gains. Fig. 9 notes slower small jobs under MIGER.	Includes mixed training/inference and multi-GPU optimization; retain only explicitly local sharing and inference-QoS evidence for assigned comparison. Abstract and introduction swap comparator order for the two JCT percentages; use section 5.2 mapping. No chart coordinates reconstructed or universal p99 claim. *Anchor:* Printed pp. 505-506, 510-511; sections 3.1-3.3, 5.1-5.4; Tables 1-3; Figs. 2, 4, 8-11; N00001.md lines 119-229, 478-662

ID	Extracted observation	Limitation and source location
S023 [41]	Sections 3.2 and 5 show optimal synchronization depends on batch and image size; the fixed strategy can impose much larger preempting-job JCT inflation after a workload change. Section 5.2 reports Deft switches to synchronizing every kernel for batch 30; earlier batch-1 case uses every 500 kernels. Table 2 cells are interleaved, so no exact violation-rate row is imported.	Preliminary six-page workshop evaluation with two principal models and one A40. Exhaustive pair profiling does not scale; no cluster/generative-model implementation claimed (section 6). Keep export year 2024 and printed conference/reference year 2023 separately; both inside window. *Anchor:* Printed pp. 15-17; sections 3.2, 4.2, 5.1-5.3 and 6; Table 2 and Fig. 4 discussed in prose; N00005.md lines 250-468
S024 [16]	Local section 5.3 states eight RNNT pods at 12% SM shares reach about 40 req/s with tail latency below 500 ms, versus one racing pod about 12 req/s and above 1250 ms. The local ResNet/RNNT comparison in Fig. 9 reports removing their observed interference with 24% shares. No unlabelled tail value is relabelled p99.	Model-specific/proportional share effects, IPC memory trust and batching limit transfer. Prose fractional-gain notation is ambiguous (3.15x with raw 296.8/71.37), so no such multiplier imported. Supplied plot coordinates/percentile identity and repetitions not reconstructed. *Anchor:* Printed pp. 641-643; sections 4, 5.1-5.4; Figs. 8-10; N00015.md lines 523-735
S025 [8]	Section 5.5 reports violation rate below 10% beyond alpha=4 versus 26% under RT-A in the discussed comparison. For short requests, jitter reductions versus Clockwork/PREMA/RT-A are 55.3/46.8/68.9% at low load and 56.0/50.3/69.3% at high load. Jitter is block-execution standard deviation, not p99 latency. Section 6 reports split overhead and static-graph limits.	One edge GPU and synthetic Poisson load; not a server-class GPU generalization. Temporal predictability sacrifices spatial parallelism; long-request stability can worsen. Table 3 split overhead and RNN limits qualify short-request improvements. *Anchor:* Printed pp. 611-613; sections 5.1-5.5, 6; Tables 1-3; Figs. 6-7; N00040.md lines 451-629

ID	Extracted observation	Limitation and source location
S026 [28]	Section 5.2 Azure: sustainable rate at 90% SLO attainment is 9.84 RPS versus LA-Dyn 4.13 and Static 1.85. Arena approximately 10.99 versus 2.37 and 2.48 RPS. These are four-GPU server outcomes; local mechanism is MIG layout control, not a single-GPU throughput multiplier.	Four-GPU evaluation and constructed layout-agnostic comparator; 90% attainment is weaker than 99% SLO targets used elsewhere. MIG hardware isolation is described, without adversarial/fault experiment. Source pixels and unreported repeats/versions are not reconstructed. *Anchor:* Printed pp. 850-853; sections 4, 5.1-5.4; Fig. 9 and Figs. 11-13; N00831.md lines 519-780
S027 [24]	Section 4.3/Fig. 9 reports mean JCT 28% lower, makespan 32% lower and normalized progress-rate STP 35% higher than MIGER. These are mixed-job/testbed metrics, not per-request p99 latency. Section 4.5 reports 1.5% MIG and 3.2% MPS reconfiguration lifecycle overhead.	Mixed inference/training and two-GPU optimization; local priority scheduling is the transferable mechanism. Many figure labels contain NUL/glyph corruption, so numerical prose only. STP/JCT does not establish online tail-latency or fault containment. *Anchor:* Article pp. 84:15-84:19; sections 4.1-4.5; Table 3; Figs. 9-12; N00839.md lines 789-1040
S028 [25]	Local section 2.4/Fig. 3 observes ParvaGPU can underperform MIG-only serving for larger BERT/ResNet50 while helping lightweight MobileNet. Section 5.2/Fig. 8 gives eight-GPU QoS-preserved throughput mean 34.61% above ParvaGPU; 56.8/62.49/34.63/44.83% above MIG-serving/gpulet/USHER/PARIS. Keep local qualitative counterexample separate from fleet improvements.	Bayesian exploration cost and instance interference qualify hybrid gains. Baseline reductions (MIG-serving fast/USHER IR only) and multi-GPU request fan-out constrain comparison. Same senior author as MIGER, but different inference-only system and experiments; not a same-study report extension identified in supplied text. *Anchor:* Printed pp. 521, 526-528; sections 2.4, 4.6, 5.1-5.4; Tables 2-3; Figs. 3, 8, 10; N00844.md lines 200-265, 700-890

ID	Extracted observation	Limitation and source location
S029 [17]	Section 4.2 reports average best-effort throughput 1.58x Static MPS with local LLaMA-7B (TP1) and 3.9x with Qwen-14B (TP2), keeping strata distinct. AdMix still increases mean TTFT/TPOT versus LLM-only; on Qwen its mean latency increases exceed Static MPS. More throughput is not zero interference.	Full six-page original workshop report; one/two-GPU workload strata and tuned-on-test-trace static reference limit transfer. Estimates of average latency do not establish p99. No testbed code rerun or security/fault experiment inferred. *Anchor:* Printed pp. 3-5; sections 3 and 4.1-4.2; Table 1; Figs. 5-6; N00872.md lines 270-453
S030 [35]	Section 9.3: versus Orion, P40 overall throughput up to 1.47x and BE up to 2.36x; A2000 up to 1.32x and 1.79x, respectively. Mean SLO attainment 99.0% across evaluated settings. Section 10 explicitly states shared-context GPU runtime errors are not isolated.	Software coloring is not a proven security boundary. MIG integration is future work; not benchmarked on A100/H100. Reverse-engineering/calibration and TVM compatibility matter; do not treat throughput maxima as averages or infer a P40 MPS result. *Anchor:* Printed pp. 276-277; sections 9.2-9.3, 10; Table 3 and Fig. 18; N00904.md lines 863-965
S031 [29]	Section 2.1/Fig. 2: co-location choice can increase energy; SDXL plus MobileNetV3 example uses 13% less energy versus no sharing despite 1.14x latency. Section 7.1/Fig. 9: 16-GPU testbed energy reduced by factors 1.4-2.3 and power by 1.3-2.6 versus baselines while meeting SLOs; fleet results are not local per-GPU savings.	Joint power/energy objectives select different allocations. Local model-pair evidence must stay distinct from fleet savings and from frequency-only effects. Source chart pixels not available; experiments/code not rerun; missing warm-up and independent-run uncertainty not assumed. *Anchor:* Printed pp. 66-67, 73-75; sections 2.1-2.2, 6, 7.1-7.2; Figs. 2, 9; N00946.md lines 140-249, 798-995

ID	Extracted observation	Limitation and source location
S032 [26]	Section 7.1/Fig. 9 reports FluidFaaS SLO hit-rate improvement of 90% for medium and 61% for heavy workload mixes over the compared systems; the abstract reports 25–75% throughput increases across workloads. These are system-level results, not a measured per-GPU utilization effect.	Pipeline, placement and MIG allocation contribute jointly; comparisons with a non-pipelined baseline cannot isolate the benefit of MIG partitioning alone. No adversarial security or fault-injection evidence is inferred from MIG descriptions. *Anchor:* N00920.md abstract, sections 3, 6, 7.1; Tables 2, 5–6; Figs. 9 and 12
S033 [19]	The introduction and evaluation report mean latency reductions of 39.1% for two BERT inference instances and 41.2% for four, under the paper's compared GPU-sharing systems. Other reported aggregate gains are not recast as p99 or universal speedups.	Quota, model pairing and GPU generation condition results. The conversion has damaged column/table layout, so no unlabeled plotted coordinates or precise tail-percentile gains were recovered. *Anchor:* N01085.md intro, sections 4, 6.1–6; Fig. 1 and BERT co-location result prose
S034 [37]	H100 Llama decode P90 time between tokens rises from 7.53 to 16.56 ms under a sleep co-tenant in section 4.2 (batch 1, 1,000-token prompt). Table 1's Green Context experiment isolates SM sets (64/68) but reports P90 TBT as high as 18.38 ms under copy-kernel memory pressure; its alone baseline is 14.2 ms on the whole GPU, so partition cost and interference cannot be collapsed into one effect.	Synthetic co-tenants and short decode traces quantify resource contention; the paper proposes a future scheduler but does not implement an end-to-end serving controller. Green Context SM partition does not partition L2 or external memory bandwidth. Table 1 conversion is interleaved; only prose and clearly attributable values are used. *Anchor:* N01096.md sections 3–5, especially 4.1–4.4; Fig. 2 and Table 1
S035 [38]	Abstract and multi-tenancy evaluation report inference throughput gains under concurrent execution; do not attribute heterogeneous placement gains solely to GPU sharing.	Edge GPU, CPU and TPU contributions are coupled; throughput is not latency predictability or security isolation. *Anchor:* N00027.md abstract; multi-tenancy and performance evaluation sections, figures on CME and DMP

ID	Extracted observation	Limitation and source location
S036 [9]	Abstract reports up to 92% throughput improvement while critical tasks incur less than 10% latency overhead in its evaluated scenarios.	Edge elastic-kernel scheduling is transferable temporal/spatial evidence, not a direct MIG/MPS comparison. *Anchor:* N00046.md abstract; sections on kernel coordination and evaluation
S037 [11]	The evaluated engine balances request goodput, throughput and latency under co-located inference; comparison is engine-level.	Compiler and priority scheduling effects cannot be isolated as pure partition effects; plotted numbers were not transcribed. *Anchor:* N00048.md abstract; section 6.2 evaluation and Fig. 1
S038 [10]	Section 5.4/Fig. 10 reports interleaved execution gains among inference jobs of up to 30% versus session-based time slicing in tested pairings.	The 19.05x p95 inference latency example in Fig. 6 uses a training co-tenant and is not evidence for two inference tenants. *Anchor:* N00049.md sections 5.1, 5.2.1 and 5.4; Figs. 6 and 10
S039 [40]	Abstract reports up to 2.40x performance versus baselines for evaluated shared-backbone transformer inference.	Shared backbone changes model computation; speedup cannot be assigned to GPU scheduling alone or generalized to unrelated models. *Anchor:* N00059.md abstract; design and experiment sections including four-tenant setup
S040 [27]	The evaluation reports SLO compliance and cost effects of orchestration under trace-driven inference; no per-GPU improvement is extracted from system-level totals.	Multiple GPUs and global scheduling contribute jointly; local mechanism claims cannot be isolated without matched per-device comparisons. *Anchor:* N00063.md abstract, sections on MIG/MPS design and evaluation of 22 workloads
S041 [39]	Section 4/Fig. 3 shows model tail latency relative to SLO while requests consolidate onto shared GPUs; power savings are whole-server effects.	The four-GPU power outcome cannot estimate per-GPU partition efficiency; MI50 setup is not a MIG evaluation. *Anchor:* N00068.md sections 4.1–4.2 and Fig. 3

ID	Extracted observation	Limitation and source location
S042 [18]	Section 5 evaluates prediction accuracy and allocation for concurrent inference with MPS; its Fig. 2 shows that MPS spatial limits do not eliminate shared-resource latency interference.	Two GPUs are present in testbed; claims here concern within-device workloads, not a two-device pooled speedup. *Anchor:* N00070.md sections 2.2 and 5.1–5.2; Fig. 2, Table 1 and testbed table
S043 [42]	Abstract reports 61–80% SLO hit-rate improvement and 47–187% cost savings over prior systems for evaluated workflows.	Figures are workflow/system results influenced by batching and scheduler search, not isolated effects of local GPU sharing. *Anchor:* N00073.md abstract, sections 3 and 5; evaluation setup and end-to-end comparison

C Completion and audit register

Before submission: reconcile four Step 5 links, canonical title/abstract and report-version rows; assess 45 queued reports and resolve 96 earlier retrieval cases; integrate or exclude the two eligible-pending reports with explicit reasons; generate a final PRISMA 2020 diagram whose boxes derive from the same ledgers; audit the single-reviewer decisions and quality scores; complete missing GPU, driver, profile, quota, arrival, warm-up and variance fields; rerun the conditional synthesis and sensitivity analysis. The published query workbook and query report provide the current receipts and historical flow, but do not eliminate these tasks.

References

[1] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like Clockwork: Performance Predictability from the Bottom Up," in *USENIX OSDI*, 2020, pp. 443–462. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/gujarati

[2] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin, "PipeSwitch: Fast Pipelined Context Switching for Deep Learning Applications," in *USENIX OSDI*, 2020, pp. 499–514. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/bai

[3] M. Han, R. Chen, W. Shen, H. Zhang, J. Yang, and H. Chen, "Real-time, Work-conserving GPU Scheduling for Concurrent DNN Inference," *ACM Transactions on Computer Systems*, vol. 44, no. 1, pp. 1–42, 2025, art. no. 1; publication date November 2025 as printed in supplied report. [Online]. Available: https://doi.org/10.1145/3768622

[4] J. Wang, Y. Wang, M. Han, and R. Chen, "SIRIUS: Colocating ML Inference and Training with Fast GPU Memory Handover," in *USENIX ATC*, 2025, pp. 1657–1675. [Online]. Available: https://www.usenix.org/conference/atc25/presentation/wang-jiali

[5] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware Fine-grained GPU Sharing for ML Applications," in *ACM EuroSys*, 2024, pp. 1075–1092. [Online]. Available: https://doi.org/10.1145/3627703.3629578

[6] W. Shen, M. Han, J. Liu, R. Chen, and H. Chen, "XSched: Preemptive Scheduling for Diverse XPUs," in *USENIX OSDI*, 2025, pp. 671–692. [Online]. Available: https://www.usenix.org/conference/osdi25/presentation/shen-weihang

[7] K. K. W. Ng, H. M. Demoulin, and V. Liu, "Paella: Low-latency Model Serving with Software-defined GPU Scheduling," in *ACM SOSP*, 2023, pp. 595–610. [Online]. Available: https://doi.org/10.1145/3600006.3613163

[8] D. Luo, T. Yu, Y. Wu, H. Wu, T. Wang, and W. Zhang, "SPLIT: QoS-Aware DNN Inference on Shared GPU via Evenly-Sized Model Splitting," in *Proceedings of ICPP 2023*, 2023, pp. 605–614. [Online]. Available: https://doi.org/10.1145/3605573.3605627

[9] Z. Zhao, N. Ling, N. Guan, and G. Xing, "Miriam: Exploiting elastic kernels for real-time multi-dnn inference on edge gpu," in *Proceedings of ACM/IEEE Symposium on Edge Computing 2023*, 2023. [Online]. Available: https://doi.org/10.1145/3625687.3625789

[10] X. Wu, J. Rao, W. Chen, and H. Huang, "Switchflow: Preemptive multitasking for deep learning," in *Proceedings of Middleware 2021*, 2021. [Online]. Available: https://doi.org/10.1145/3464298.3493391

[11] L. Ma, H. Chen, E. Shao, L. Wang, Q. Chen, and G. Tan, "Elasticroom: Multi-tenant dnn inference engine via co-design with resource-constrained compilation and strong priority scheduling," in *Proceedings of HPDC 2024*, 2024. [Online]. Available: https://doi.org/10.1145/3625549.3658654

[12] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing," in *USENIX ATC*, 2022, pp. 199–216. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/choi-seungbeom

[13] A. Dhakal, S. G. Kulkarni, and K. K. Ramakrishnan, "GSLICE: Controlled Spatial Sharing of GPUs for a Scalable Inference Platform," in *ACM SoCC*, 2020, pp. 492–506. [Online]. Available: https://doi.org/10.1145/3419111.3421284

[14] L.-X. Ma, L.-P. Wang, E. Shao, R.-Y. Cao, and G.-M. Tan, "VastPipe: A High-Throughput Inference System via Adaptive Space-Division Multiplexing for Diverse Accelerators," *Journal of Computer Science and Technology*, vol. 40, no. 2, pp. 444–463, 2025. [Online]. Available: https://doi.org/10.1007/s11390-024-3773-5

[15] F. Xu, J. Xu, J. Chen, L. Chen, R. Shang, Z. Zhou, and F. Liu, "iGniter: Interference-Aware GPU Resource Provisioning for Predictable DNN Inference in the Cloud," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 3, pp. 812–827, 2023. [Online]. Available: https://doi.org/10.1109/TPDS.2022.3232715

[16] J. Gu, Y. Zhu, P. Wang, M. Chadha, and M. Gerndt, "FaST-GShare: Enabling Efficient Spatio-Temporal GPU Sharing in Serverless Computing for Deep Learning Inference," in *Proceedings of ICPP 2023*, 2023, pp. 635–644. [Online]. Available: https://doi.org/10.1145/3605573.3605638

[17] S. Li, T. Hanawa, and Y. Miki, "Adaptive GPU Sharing for Real-time LLM Serving with Best-effort Workloads," in *ICPP Workshops 2026*, 2026, pp. 1–6. [Online]. Available: https://doi.org/10.1145/3816891.3834895

[18] Y. Chen, W. Li, H. Zhou, X. Yang, and Y. Yin, "Deinfer: A gpu resource allocation algorithm with spatial sharing for near-deterministic inferring tasks," in *Proceedings of ICPP 2024*, 2024. [Online]. Available: https://doi.org/10.1145/3673038.3673091

[19] S. Zhang, Q. Chen, W. Cui, H. Zhao, C. Xue, Z. Zheng, W. Lin, and M. Guo, "Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing," in *Proceedings of EuroSys 2025*, 2025. [Online]. Available: https://doi.org/10.1145/3689031.3696070

[20] C. Savard, N. Manganelli, B. Holzman, L. Gray, A. Perloff, K. Pedro, K. Stenson, and K. Ulmer, "Optimizing High-Throughput Inference on Graph Neural Networks at Shared Computing Facilities with the NVIDIA Triton Inference Server," *Computing and Software for Big Science*, vol. 8, 2024, art. no. 14. [Online]. Available: https://link.springer.com/article/10.1007/s41781-024-00123-2

[21] T. Adufu and Y. Kim, "Dyna-P: placement-aware dynamic partitioning for lightweight applications with modern GPUs," *Cluster Computing*, vol. 28, 2025, art. no. 608. [Online]. Available: https://doi.org/10.1007/s10586-025-05284-2

[22] M. Lee, S. Seong, M. Kang, J. Lee, G.-J. Na, I.-G. Chun, D. S. Nikolopoulos, and C.-H. Hong, "ParvaGPU: Efficient Spatial GPU Sharing for Large-Scale DNN Inference in Cloud Environments," in *IEEE/ACM SC*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10793202

[23] B. Zhang, S. Li, and Z. Li, "MIGER: Integrating Multi-Instance GPU and Multi-Process Service for Deep Learning Clusters," in *Proceedings of ICPP 2024*, 2024, pp. 504–513. [Online]. Available: https://doi.org/10.1145/3673038.3673089

[24] Z. Jiang, Z. Cai, Y. Li, Z. Wang, R. Ma, H. Guan, and R. Buyya, "MMK: A Hybrid Scheduling Framework for Fine-Grained GPU Sharing for Deep Learning Applications," *ACM Transactions on Architecture and Code Optimization*, vol. 23, no. 3, pp. 1–25, 2026, art. no. 84. [Online]. Available: https://doi.org/10.1145/3820163

[25] B. Zhang, Y. Wang, and Z. Li, "BOER: Enhancing Resource Utilization for Deep Learning Inference with Hybrid Spatial GPU Sharing," in *Proceedings of SC 2025*, 2025, pp. 519–532. [Online]. Available: https://doi.org/10.1145/3712285.3759857

[26] X. Hui, Y. Xu, and X. Shen, "FluidFaaS: A Dynamic Pipelined Solution for Serverless Computing with Strong Isolation-based GPU Sharing," in *Proceedings of HPDC 2025*, 2025. [Online]. Available: https://doi.org/10.1145/3731545.3731580

[27] V. M. Bhasi, A. Sharma, R. Jain, J. R. Gunasekaran, A. Pattnaik, M. T. Kandemir, and C. Das, "Towards slo-compliant and cost-effective serverless computing on emerging gpu architectures," in *Proceedings of Middleware 2024*, 2024. [Online]. Available: https://doi.org/10.1145/3652892.3700760

[28] J. Chen, Y. Liang, B. Gao, Z. Zhou, F. Xu, X. Chen, and D. Guo, "MIGServe: Layout-Aware Multi-Instance GPU Management for Efficient LLM Serving," in

Proceedings of ICPP 2026, 2026, pp. 844–854. [Online]. Available: https://doi.org/10.1145/3832810.3832898

[29] P. Sinha, D. Liakopoulos, N. Lemma, and N. J. Yadwadkar, "Beyond Utilization: Energy-Conscious GPU Sharing for Inference Serving," in *Proceedings of SOSP 2026*, 2026, pp. 65–89. [Online]. Available: https://doi.org/10.1145/3830418.3843913

[30] M. Yu, A. Wang, D. Chen, H. Yu, X. Luo, Z. Li, W. Wang, R. Chen, D. Nie, H. Yang, and Y. Ding, "Torpor: GPU-Enabled Serverless Computing for Low-Latency, Resource-Efficient Inference," in *USENIX ATC*, 2025, pp. 597–612. [Online]. Available: https://www.usenix.org/conference/atc25/presentation/yu

[31] H. Wu, Y. Yu, J. Deng, S. Ibrahim, S. Wu, H. Fan, Z. Cheng, and H. Jin, "StreamBox: A Lightweight GPU SandBox for Serverless Inference Workflow," in *USENIX ATC*, 2024, pp. 59–73. [Online]. Available: https://www.usenix.org/conference/atc24/presentation/wu-hao

[32] S. Zhang, A. Xu, Q. Chen, H. Zhao, W. Cui, Z. Wang, Y. Li, L. Xiao, and M. Guo, "KRYPTON: Efficient Performance-Aware GPU Sharing with Compatibility and Isolation through Kernel Space Interception," in *USENIX ATC*, 2025, pp. 1003–1019. [Online]. Available: https://www.usenix.org/conference/atc25/presentation/zhang-shulai

[33] W. Zhao, A. Jayarajan, and G. Pekhimenko, "Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads," in *ACM ASPLOS Volume 1*, 2025, pp. 1052–1068. [Online]. Available: https://doi.org/10.1145/3669940.3707282

[34] J. Lou, Y. Sun, J. Zhang, H. Cao, Y. Zhang, and N. Sun, "ArkGPU: enabling applications' high-goodput co-location execution on multitasking GPUs," *CCF Transactions on High Performance Computing*, vol. 5, pp. 304–321, 2023. [Online]. Available: https://doi.org/10.1007/s42514-023-00154-y

[35] Y. Zhang, H. Yu, C. Han, C. Wang, B. Lu, Y. Li, Z. Jiang, Y. Li, X. Chu, and H. Li, "SGDRC: Software-Defined Dynamic Resource Control for Concurrent DNN Inference on NVIDIA GPUs," in *Proceedings of PPoPP 2025*, 2025, pp. 267–281. [Online]. Available: https://doi.org/10.1145/3710848.3710863

[36] Y. Peng and H. Peng, "InferFair: Towards QoS-aware scheduling for performance isolation guarantee in heterogeneous model serving systems," *Future Generation Computer Systems*, vol. 150, pp. 10–20, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X23003187

[37] P. Elvinger, F. Strati, N. E. Jerger, and A. Klimovic, "Understanding GPU Resource Interference One Level Deeper," in *Proceedings of SoCC 2025*, 2025, printed SoCC 2025 event; export year 2026. [Online]. Available: https://doi.org/10.1145/3772052.3772270

[38] J. Hao, P. Subedi, L. Shramaswamy, and I. K. Kim, "Reaching for the sky: Maximizing deep learning inference throughput on edge devices with ai multi-tenancy," *ACM Transactions on Internet Technology*, 2023. [Online]. Available: https://doi.org/10.1145/3546192

[39] A. Jahanshahi, M. Rezvani, and D. Wong, "Wattwiser: Power & resource-efficient scheduling for multi-model multi-gpu inference servers," in *Proceedings of IGSC 2023*, 2023. [Online]. Available: https://doi.org/10.1145/3634769.3634807

[40] Y. Zhang, Z. Zhang, W. Bao, and D. Yuan, "Itif: Integrated transformers inference framework for multiple tenants on gpu," in *Proceedings of ICPP 2023*, 2023. [Online]. Available: https://doi.org/10.1145/3605573.3605585

[41] Z. Xia, Y. Hao, J. Duan, C. Wang, and J. Jiang, "Towards Optimal Preemptive GPU Time-Sharing for Edge Model Serving," in *Proceedings of the 9th International Workshop on Container Technologies and Container Clouds (WoC 2023)*, 2024, pp. 13–18, woC 2023 event; export publication year 2024; printed reference year 2023. [Online]. Available: https://doi.org/10.1145/3631311.3632401

[42] X. Hui, Y. Xu, Z. Guo, and X. Shen, "Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus," in *Proceedings of HPDC 2024*, 2024. [Online]. Available: https://doi.org/10.1145/3625549.3658657

[43] M. Chow, A. Jahanshahi, and D. Wong, "KRISP: Enabling Kernel-wise RIght-sizing for Spatial Partitioned GPU Inference Servers," in *IEEE HPCA*, 2023, pp. 624–637. [Online]. Available: https://doi.org/10.1109/HPCA56546.2023.10071121